

Universidad Tecnológica Nacional
Facultad Regional San Rafael
Ingeniería en Sistemas de Información

Software Metrics & Estimation

Trabajo Práctico Integrador · Ingeniería y Calidad de Software 2026

Plan de trabajo, distribución de tareas y Product Backlog inicial

Sprint 0 a Sprint 4

Integrantes

López, Juan Pablo
Piastrellini, Mariano
Bravo, Carolina

Versión 1.1 · 26/09/2026

Complementado por la **Guía de desarrollo**. Lo marcado **[A CONFIRMAR]** sigue abierto (sección 10).

1. Qué pide la consigna (resumen operativo)

Desarrollar una aplicación que permita **administrar un proyecto de software** y obtener información de su **estimación, planificación, seguimiento y calidad**. Puede ser web, escritorio o consola, pero **el núcleo y las reglas de negocio deben estar en Go**. Nuestra decisión: **aplicación web**.

Obligatorio en el proceso

Práctica	Qué implica concretamente para nosotros
Go	Todo el dominio, cálculos, validaciones y API en Go. La UI puede ser HTML servido por Go.
Scrum	Roles (Product Architect = profesores, Agile Enabler = 1 integrante, Product Builders = todos), Product Backlog, Sprints, Planning, Daily, Review y Retrospective con acta.
SDD	Cada funcionalidad principal tiene una especificación escrita antes del código, versionada en el repo (/specs), con: objetivo, entradas, salidas, reglas de negocio, restricciones, casos límite, errores y criterios de aceptación.
BDD	Escenarios Given–When–Then (en español con # language: es) que cubran casos normales, alternativos, límite y de error; automatizados con godog siempre que se pueda (/features).
TDD	Ciclo RED → GREEN → REFACTOR para métricas, estimación, reglas de negocio y validaciones. La evidencia es el historial de commits : un commit por fase.
Git	Historial de contribuciones de los tres (cada uno commitea con su cuenta), ramas por historia, Pull Requests revisados.
IA	Permitida para requisitos, specs, código, tests, refactor, revisión y documentación. Todo lo generado debe ser entendido, revisado y validado por nosotros (lo registramos en docs/ia/registro.md).

Requerimientos funcionales mínimos → épicas

#	Requerimiento de la consigna	Épica	Responsable
1	Gestión de proyectos: crear/modificar, integrantes, fechas, estado	E1 Proyectos	Juan Pablo
2	Product Backlog: id, título, descripción, prioridad, estado, SP, criterios de aceptación	E2 Backlog	Mariano
3	Sprints: crear, Sprint Goal, asignar historias, completadas, cerrar, consultar anteriores	E3 Sprints	Juan Pablo
4	Estimación con Story Points + Planning Poker (votos ocultos, revelar, diferencias, rondas, acordada)	E4 Estimación	Mariano
5	Registro de esfuerzo: integrante, fecha, actividad, horas (estimado vs real)	E5 Esfuerzo	Carolina
6	Defectos: descripción, severidad, estado, historia, sprint detección y resolución	E6 Defectos	Carolina
7	Métricas: SP planificados/completados, velocidad, horas, desviación, % completado, defectos	E7 Métricas	Carolina
8	Dashboard con estado del proyecto y gráficos	E8 Dashboard	Carolina
9	Reportes de proyecto o sprint (+ exportación a PDF en Sprint 4)	E9 Reportes	Juan Pablo

Criterios de evaluación y cómo los atacamos

Criterio	Peso	Estrategia
Producto funcional	25 %	Deploy continuo en Vercel desde el Sprint 1; cada historia se considera terminada sólo si está en producción.
SDD, BDD y TDD	25 %	Cadena de trazabilidad obligatoria por historia (sección 4) y Definition of Done que la exige.
Calidad del software	20 %	Arquitectura por capas, dominio puro sin dependencias, errores tipados, linter y cobertura en CI.
Gestión del proyecto	20 %	GitHub Projects con iteraciones, SP y prioridades; actas de Review/Retro en el repo; PRs con revisión.
Equipo y presentación	10 %	Cada integrante es dueño de épicas de punta a punta y las defiende en la demo final.

2. Decisiones técnicas propuestas

Aspecto	Propuesta	Por qué
Lenguaje	Go (última versión estable), módulo único	Exigido. Un solo módulo simplifica imports y el deploy.
Web / API	net/http + router chi; handlers JSON y HTML	Estándar, liviano, fácil de testear con httptest.
Interfaz	HTML con html/template + HTMX + Bootstrap 5 + Chart.js para gráficos	Decidido: todo en Go, sin un segundo stack (React/Next.js). El equipo no tiene experiencia previa y los sprints son de una semana.
Base de datos	Supabase (PostgreSQL) — un proyecto dev y otro prod (el plan gratis permite 2)	Gratis, Postgres real, panel web, Auth incluido.
Acceso a datos	pgx/v5 + migraciones versionadas con goose o golang-migrate (/migrations)	SQL explícito; las migraciones también son trazables en Git.
Deploy	Vercel con runtime Go: una única función api/index.go que monta el router; vercel.json redirige todo a ella	Previews automáticos por PR. Plan B si hay límites: Render / Fly.io / Railway.
Autenticación	Supabase Auth (email + contraseña); middleware Go valida el JWT	Necesario para que en Planning Poker cada integrante vote desde su dispositivo.
Tests unitarios	testing de la stdlib, tablas de casos, -race, -coverprofile	Lo que pide la consigna, sin magia.
BDD	github.com/cucumber/godog, features en español	Ejecuta los .feature como tests en CI.
PDF (Sprint 4)	github.com/johnfercher/maroto/v2 (alternativa go-pdf/fpdf)	Genera el reporte en Go puro dentro de la función serverless.
Calidad	gofmt, go vet, golangci-lint, GitHub Actions, protección de main	Evidencia objetiva de calidad para el informe final.

Advertencias técnicas importantes (para no perder tiempo después)

- **Vercel es serverless:** no hay memoria compartida entre requests ni WebSockets. El estado del Planning Poker (votos, rondas) vive en la base de datos y la pantalla se actualiza con **polling de HTMX** (hx-trigger="every 2s").
- **Supabase + serverless:** conectarse por el pooler (puerto 6543, modo transacción) y configurar pgx en modo QueryExecModeSimpleProtocol (el pooler en modo transacción no soporta prepared statements cacheados).
- **Supabase gratis pausa el proyecto tras ~7 días sin actividad.** Para uso personal posterior conviene un ping periódico o reactivarlo a mano.
- **Nunca usar "Squash and merge"** en los PRs: borraría los commits RED/GREEN/REFACTOR que son la evidencia de TDD. Usar merge commit o rebase.
- Fechas de esfuerzo como DATE (no timestamp) y zona horaria America/Argentina/Mendoza para evitar corrimientos de día.

2.1 Arquitectura y estructura del repositorio

Arquitectura por capas (estilo hexagonal): el **dominio** no conoce ni la base de datos ni HTTP, por eso sus tests son rápidos y puros (TDD). Los **servicios** orquestan casos de uso contra interfaces de repositorio; en los tests se usan repositorios en memoria (fakes).

```
software-metrics/
├── api/index.go           # endpoint de Vercel: monta el router
├── cmd/server/main.go     # servidor local (go run ./cmd/server)
├── internal/
│   ├── domain/           # entidades + reglas de negocio + validaciones (sin dependencias)
│   │   ├── project/ backlog/ sprint/ estimation/ effort/ defect/
│   ├── metrics/          # cálculos puros de métricas (TDD intensivo)
│   ├── service/          # casos de uso; dependen de interfaces de repositorio
│   ├── store/postgres/   # implementación de repositorios con pgx
│   ├── store/memory/     # fakes en memoria para tests
│   ├── http/             # router chi, handlers, middleware (auth, errores)
│   └── report/           # armado de reportes y exportación PDF
├── web/templates/ web/static/ # vistas HTML (embed.FS), CSS, JS (HTMX, Chart.js)
├── migrations/           # 0001_init.sql, 0002_... (versionadas)
├── specs/                # SDD: HU-xx-nombre.md (una spec por historia)
├── features/             # BDD: hu-xx-nombre.feature + steps en Go
├── docs/
│   ├── scrum/ (planning, reviews, retros/sprint-N.md, dailies)
│   ├── ia/registro.md    # qué se generó con IA y cómo se validó
│   ├── arquitectura.md manual-usuario.md informe-metricas-cobertura.md
├── .github/ (workflows/ci.yml, ISSUE_TEMPLATE/, pull_request_template.md)
├── go.mod Makefile vercel.json README.md
```

3. Roles y distribución del trabajo

Rol Scrum	Quién	Responsabilidades
Product Architect	Profesores (Cliente)	Presentan la visión, validan historias y aceptan incrementos en las Reviews.
Agile Enabler	López, Juan Pablo	Facilita Planning, Daily, Review y Retro; mantiene el tablero; redacta actas; vigila la Definition of Done y los impedimentos. También es Product Builder.
Product Builder	Los tres	Cada uno es dueño de punta a punta de sus épicas: historia → spec → CA → BDD → tests → código → deploy.

Criterio de división: por épicas verticales (no por capas), para que los tres apliquen SDD, BDD y TDD, tengan commits en todas las capas y puedan defender su parte en la presentación. La carga en story points quedó equilibrada (ver tabla de totales al final del backlog). Las tareas propias del Agile Enabler no se cuentan en story points; por eso Juan Pablo tiene el Sprint 4 más liviano en desarrollo.

Integrante	Épicas / ítems técnicos	Transversal
López, Juan Pablo	E1 Proyectos e integrantes · E3 Sprints · E9 Reportes y PDF · TEC-01 Arquitectura · TEC-02 CI	Agile Enabler, tablero, actas, coordinación de la presentación final
Piastrellini, Mariano	E2 Product Backlog · E4 Estimación y Planning Poker · TEC-03 Base de datos · TEC-05 Autenticación · TEC-06 Tests de integración	Documentación técnica (arquitectura, API, deploy)
Bravo, Carolina	E5 Esfuerzo · E6 Defectos · E7 Métricas · E8 Dashboard · TEC-04 Deploy Vercel · TEC-07 Layout de la UI	Manual de usuario e informe de métricas y cobertura

Revisión cruzada de código (rotativa y obligatoria)

Todo PR necesita **1 aprobación** de otra persona antes de mergear: **Juan Pablo revisa a Mariano, Mariano revisa a Carolina y Carolina revisa a Juan Pablo**. Así todos conocen el código de los demás (clave para la defensa oral).

Dependencias entre integrantes (acordar en Sprint 0)

- **Modelo de dominio y esquema de BD compartido:** se define entre los tres en el Sprint 0 (entidades Proyecto, Integrante, Historia, Sprint, SesiónPoker, Voto, Tarea, RegistroEsfuerzo, Defecto). Mariano lo implementa como migración inicial.
- **Métricas (Carolina)** consume datos de Backlog (Mariano), Sprints (Juan Pablo) y Esfuerzo/Defectos (Carolina). Para no bloquearse, las funciones de `internal/metrics` reciben **structs simples** y se testean con TDD desde el día 1; la integración con repositorios llega después.
- **Reportes (Juan Pablo)** reutiliza las funciones de métricas de Carolina: no se duplican cálculos.
- **Planning Poker (Mariano)** necesita identidad de usuario → Autenticación (TEC-05) antes de HU-18.

4. Metodología por historia (cadena de trazabilidad)

La consigna exige demostrar el recorrido **Historia de Usuario** → **Especificación SDD** → **Criterios de Aceptación** → **Escenarios BDD** → **Tests** → **Código Go**. Lo aplicamos a **todas** las historias funcionales con el mismo identificador en cada artefacto:

Paso	Artefacto	Dónde vive	Identificador
1. Historia de usuario	Issue de GitHub (plantilla HU)	GitHub Projects	HU-31
2. Especificación SDD	Markdown con las 8 secciones obligatorias	specs/HU-31-velocidad.md	SPEC-31
3. Criterios de aceptación	Lista numerada en la spec y en el issue	spec + issue	CA-31.1 ... CA-31.n
4. Escenarios BDD	Gherkin en español (normal, alternativo, límite, error)	features/hu-31-velocidad.feature	tag @HU-31
5. Tests	Unitarios (TDD) + steps de godog	internal/metrics/velocity_test.go	comentario // CA-31.2
6. Código Go	Implementación mínima que pasa los tests	internal/metrics/velocity.go	commits (HU-31)
7. PR + deploy	PR que cierra el issue; preview en Vercel	GitHub	Closes #n

4.1 Convenciones de Git (evidencia de TDD)

- Ramas: feature/HU-31-velocidad, fix/DEF-4-..., chore/TEC-02-ci. main protegida: sólo por PR con CI verde y 1 aprobación.
- Commits con Conventional Commits y la fase de TDD explícita:

```
test(HU-31): RED - velocidad promedio los últimos N sprints cerrados      <- el test falla
feat(HU-31): GREEN - implementa Velocidad con ventana configurable        <- mínimo para pasar
refactor(HU-31): extrae validación de ventana y ordena sprints            <- tests siguen verdes
test(HU-31): RED - error cuando la ventana es menor o igual a cero
feat(HU-31): GREEN - devuelve ErrVentanaInvalida
docs(HU-31): spec SDD y escenarios BDD de velocidad
```

4.2 Definition of Ready y Definition of Done

Definition of Ready (para entrar al sprint)	Definition of Done (para pasar a Done)
- Historia en formato Como / Quiero / Para - Criterios de aceptación acordados con el cliente - Estimada en story points (Planning Poker) - Dependencias identificadas - Responsable asignado	- Spec SDD completa en /specs y revisada - Escenarios BDD (normal, alternativo, límite, error) pasando - Tests unitarios escritos con TDD (commits RED/GREEN/REFACTOR) - Cobertura del paquete de dominio $\geq 80\%$ - go vet y golangci-lint sin errores - PR revisado y aprobado por otro integrante - Desplegado en Vercel y CA verificados en la app - Uso de IA registrado (si hubo)

5. Ejemplo completo de trazabilidad: HU-31 Velocidad del equipo

Este ejemplo sirve de **plantilla** para el resto de las historias y es buen candidato para la demostración final. El código es un punto de partida: se construye con TDD (primero el test en rojo).

Paso 1 — Historia de usuario

HU-31 · Velocidad del equipo · 3 SP · Prioridad Alta · Responsable: Carolina

- **Como** Agile Enabler **quiero** conocer la velocidad del equipo **para** decidir cuántos story points comprometer en el próximo sprint.

Paso 2 — Especificación SDD (specs/HU-31-velocidad.md)

Sección	Contenido
Objetivo	Calcular la velocidad del equipo como promedio de story points completados en los últimos N sprints cerrados.
Entradas	Lista de sprints cerrados (número, SP completados); ventana N (entero, por defecto 3).
Salidas esperadas	Velocidad (decimal, redondeada a 1 decimal en la UI).
Reglas de negocio	RN1: sólo cuentan sprints en estado Cerrado. RN2: se toman los N de mayor número. RN3: si hay menos de N, se promedian los existentes. RN4: el orden de entrada no afecta el resultado.
Restricciones	Cálculo puro en <code>internal/metrics</code> , sin acceso a BD. N configurable por proyecto.
Casos límite	Sin sprints cerrados → 0. Un solo sprint → su valor. N mayor que la cantidad de sprints. Sprints con 0 SP completados.
Condiciones de error	$N \leq 0 \rightarrow \text{ErrVentanaInvalida}$. SP completados negativos → <code>ErrSPNegativos</code> .
Criterios de aceptación	CA-31.1 a CA-31.5 (paso 3).

Paso 3 — Criterios de aceptación

- **CA-31.1** La velocidad es el promedio de SP completados en los últimos N sprints cerrados.
- **CA-31.2** Si hay menos de N sprints cerrados, se promedian los existentes.
- **CA-31.3** Si no hay sprints cerrados, la velocidad es 0 y se informa "Aún no hay sprints cerrados".
- **CA-31.4** Los sprints activos o planificados no se consideran.
- **CA-31.5** Una ventana $N \leq 0$ produce un error descriptivo.

Paso 4 — Escenarios BDD (features/hu-31-velocidad.feature)

```
# language: es
@HU-31
Característica: Velocidad del equipo
  Como Agile Enabler
  Quiero conocer la velocidad del equipo
  Para decidir cuántos story points comprometer en el próximo sprint

Esquema del escenario: Promedio de los últimos sprints cerrados (CA-31.1, CA-31.2)
  Dado un proyecto con una ventana de velocidad de <ventana> sprints
  Y sprints cerrados que completaron "<completados>" story points
  Cuando consulto la velocidad del equipo
  Entonces la velocidad informada es <velocidad>

Ejemplos:
  | ventana | completados | velocidad |
  | 3       | 10,20,30,40 | 30        |
  | 3       | 10,20       | 15        |
  | 1       | 12,8        | 8         |

Escenario: Proyecto sin sprints cerrados (CA-31.3)
  Dado un proyecto con una ventana de velocidad de 3 sprints
  Y que el proyecto no tiene sprints cerrados
  Cuando consulto la velocidad del equipo
  Entonces la velocidad informada es 0
  Y se muestra el mensaje "Aún no hay sprints cerrados"

Escenario: Ventana de cálculo inválida (CA-31.5)
  Dado un proyecto con una ventana de velocidad de 0 sprints
  Cuando consulto la velocidad del equipo
  Entonces se informa el error "la ventana de sprints debe ser mayor a cero"
```

Paso 5 — Tests unitarios escritos primero (internal/metrics/velocity_test.go)

```
package metrics_test

import (
    "errors"
    "testing"

    "github.com/<org>/software-metrics/internal/metrics"
)

func TestVelocidad(t *testing.T) {
    casos := []struct {
        nombre string
        sprints []metrics.SprintCerrado
        n        int
        want     float64
        wantErr  error
    }{
        {"promedia los últimos N (CA-31.1)", []metrics.SprintCerrado{{1, 10}, {2, 20}, {3, 30}, {4, 40}}, 3, 30, nil},
        {"menos sprints que la ventana (CA-31.2)", []metrics.SprintCerrado{{1, 10}, {2, 20}}, 3, 15, nil},
        {"sin sprints cerrados (CA-31.3)", nil, 3, 0, nil},
        {"el orden de entrada no importa (RN4)", []metrics.SprintCerrado{{3, 30}, {1, 10}, {2, 20}}, 2, 25, nil},
        {"ventana inválida (CA-31.5)", []metrics.SprintCerrado{{1, 10}}, 0, 0, metrics.ErrVentanaInvalida},
        {"story points negativos", []metrics.SprintCerrado{{1, -5}}, 3, 0, metrics.ErrSPNegativos},
    }
    for _, c := range casos {
        t.Run(c.nombre, func(t *testing.T) {
            got, err := metrics.Velocidad(c.sprints, c.n)
            if !errors.Is(err, c.wantErr) {
                t.Fatalf("error = %v, se esperaba %v", err, c.wantErr)
            }
            if got != c.want {
                t.Errorf("velocidad = %v, se esperaba %v", got, c.want)
            }
        })
    }
}
```

Paso 6 — Código Go que hace pasar los tests (internal/metrics/velocity.go)

```
package metrics

import (
    "cmp"
    "errors"
    "slices"
)

var (
    ErrVentanaInvalida = errors.New("la ventana de sprints debe ser mayor a cero")
    ErrSPNegativos     = errors.New("los story points completados no pueden ser negativos")
)

// SprintCerrado resume el resultado de un sprint ya cerrado.
type SprintCerrado struct {
    Numero      int
    SPCompletados int
}

// Velocidad devuelve el promedio de story points completados en los últimos n sprints cerrados.
func Velocidad(sprints []SprintCerrado, n int) (float64, error) {
    if n <= 0 {
        return 0, ErrVentanaInvalida
    }
    if len(sprints) == 0 {
        return 0, nil
    }
    ordenados := slices.Clone(sprints)
    slices.SortFunc(ordenados, func(a, b SprintCerrado) int { return cmp.Compare(a.Numero, b.Numero) })
    n = min(n, len(ordenados))

    total := 0
    for _, s := range ordenados[len(ordenados)-n:] {
        if s.SPCompletados < 0 {
            return 0, ErrSPNegativos
        }
        total += s.SPCompletados
    }
    return float64(total) / float64(n), nil
}
```

Los steps de godog (features/steps/velocity_steps_test.go) traducen cada frase del .feature a llamadas a metrics.Velocidad o al servicio, y se ejecutan con go test ./features/... en el CI.

6. Product Backlog inicial

Borrador para validar **con el cliente** en el Sprint 0. Cada ítem se carga como issue en GitHub Projects con los campos Épica, Story Points, Prioridad, Sprint (iteración) y Responsable. Los story points son una **primera estimación** (escala Fibonacci) que el equipo debe re-estimar con Planning Poker en la Planning. Prioridad: A = Alta, M = Media, B = Baja.

Ítems técnicos (habilitadores)

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
TEC-01	Esqueleto del proyecto y arquitectura por capas Como equipo queremos una estructura de proyecto Go por capas para desarrollar en paralelo sin pisarnos. — go.mod, estructura de carpetas de la sección 2.1, Makefile (make test, make run, make lint) — servidor local y handler de Vercel comparten el mismo router — README con cómo levantar el proyecto	3	A	1	Juan Pablo
TEC-02	Integración continua Como equipo queremos que cada PR ejecute automáticamente las validaciones de calidad para no romper main. — GitHub Actions ejecuta gofmt, go vet, golangci-lint, go test -race -cover y godog — el PR no se puede mergear con el CI en rojo — el porcentaje de cobertura queda publicado como artefacto	2	A	1	Juan Pablo
TEC-03	Base de datos y migraciones Como equipo queremos una base PostgreSQL en Supabase con migraciones versionadas para persistir los datos. — proyectos Supabase dev y prod creados; credenciales sólo en variables de entorno (nunca en el repo) — migración inicial con el modelo acordado en Sprint 0 — repositorios con pgx detrás de interfaces del dominio	3	A	1	Mariano
TEC-04	Deploy en Vercel Como equipo queremos desplegar automáticamente la aplicación para que el cliente pueda probar cada incremento. — cada merge a main despliega a producción y cada PR genera un preview — variables de entorno configuradas por ambiente — endpoint /health responde 200 y verifica la conexión a la BD	3	A	1	Carolina
TEC-05	Autenticación de usuarios Como integrante quiero iniciar sesión para que el sistema sepa quién registra horas y quién vota. — registro e inicio de sesión con Supabase Auth — middleware Go valida el JWT y rechaza con 401 si es inválido o expiró — cada usuario se vincula con un integrante del proyecto	5	A	2	Mariano
TEC-06	Pruebas de integración y umbral de cobertura Como equipo queremos probar los repositorios contra una base real y exigir cobertura mínima para sostener la calidad. — tests de repositorios contra la base de dev (o Postgres en Docker) con datos de prueba aislados — el CI falla si la cobertura de internal/domain y internal/metrics baja del 80 %	3	M	3	Mariano
TEC-07	Layout base de la interfaz Como usuario quiero una interfaz consistente y navegable para usar la aplicación sin capacitación. — plantilla base con navegación, mensajes de error y confirmaciones — diseño usable en celular y escritorio — componentes HTMX reutilizables (formularios, tablas, modales)	3	A	2	Carolina

E1 Proyectos

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-01	Crear proyecto Como Agile Enabler quiero crear un proyecto con nombre, descripción, fecha de inicio y de finalización para comenzar a gestionarlo. - CA-01.1 nombre obligatorio (3 a 100 caracteres) y único - CA-01.2 la fecha de fin debe ser posterior a la de inicio - CA-01.3 el proyecto se crea en estado Planificado - CA-01.4 ante un dato inválido se indica qué campo falló	3	A	1	Juan Pablo
HU-02	Modificar proyecto Como Agile Enabler quiero modificar los datos de un proyecto para mantener su información actualizada. - CA-02.1 aplican las mismas validaciones que al crear - CA-02.2 la fecha de fin no puede quedar antes del fin del último sprint - CA-02.3 un proyecto Finalizado no se puede modificar	2	M	2	Juan Pablo
HU-03	Registrar integrantes Como Agile Enabler quiero registrar integrantes con nombre, email y rol para asignarles trabajo y registrar su esfuerzo. - CA-03.1 email válido y no repetido dentro del proyecto - CA-03.2 rol: Agile Enabler o Product Builder; un solo Agile Enabler por proyecto - CA-03.3 un integrante con esfuerzo registrado no se elimina: se da de baja	3	A	1	Juan Pablo
HU-04	Consultar estado del proyecto Como integrante quiero consultar el estado del proyecto para saber cómo avanza. - CA-04.1 muestra estado (Planificado / En curso / Finalizado), sprint activo, días restantes y % de SP completados - CA-04.2 el estado se deriva de las fechas y los sprints según la regla de la spec - CA-04.3 un proyecto sin sprints se muestra como Planificado	3	A	2	Juan Pablo

E2 Backlog

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-05	Crear ítem del Product Backlog Como Product Builder quiero agregar historias al backlog con título, descripción, prioridad y criterios de aceptación para registrar el trabajo pendiente. - CA-05.1 identificador autogenerado y correlativo por proyecto (HU-1, HU-2, ...) - CA-05.2 título obligatorio; prioridad Alta, Media o Baja - CA-05.3 estado inicial Pendiente; story points opcionales al crear	3	A	1	Mariano
HU-06	Editar y eliminar ítem Como Product Builder quiero editar o eliminar historias para mantener el backlog ordenado. - CA-06.1 editar no cambia el identificador - CA-06.2 no se eliminan historias Hechas ni asignadas a un sprint cerrado - CA-06.3 eliminar pide confirmación	2	M	1	Mariano
HU-07	Gestionar criterios de aceptación Como Product Builder quiero cargar y marcar como cumplidos los criterios de aceptación de una historia para verificar que está terminada. - CA-07.1 una historia necesita al menos un CA para entrar a un sprint - CA-07.2 cada CA se marca cumplido o no cumplido - CA-07.3 se muestra el progreso (por ejemplo 2/3)	2	A	1	Mariano
HU-08	Cambiar estado de una historia Como Product Builder quiero cambiar el estado de una historia para reflejar su avance. - CA-08.1 transiciones válidas: Pendiente → En sprint → En progreso → Hecho (y vuelta a Pendiente al cerrar el sprint) - CA-08.2 sólo pasa a Hecho si todos sus CA están cumplidos - CA-08.3 una transición inválida devuelve un error descriptivo	3	A	1	Mariano

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-09	Priorizar el backlog Como equipo queremos ordenar el backlog por prioridad para decidir qué entra en el próximo sprint. – CA-09.1 orden por prioridad y orden manual dentro de la misma prioridad – CA-09.2 el orden se persiste – CA-09.3 las historias Hechas se muestran aparte	3	M	2	Mariano
HU-10	Filtrar y buscar historias Como integrante quiero filtrar y buscar historias para encontrarlas rápido. – CA-10.1 filtros por estado, prioridad y texto del título, combinables – CA-10.2 sin resultados se muestra un mensaje claro	2	B	2	Mariano

E3 Sprints

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-11	Crear sprint con Sprint Goal Como Agile Enabler quiero crear un sprint con fechas y Sprint Goal para planificar la iteración. – CA-11.1 Sprint Goal obligatorio – CA-11.2 fechas dentro del rango del proyecto y sin superponerse con otro sprint – CA-11.3 número de sprint correlativo; estado inicial Planificado	3	A	1	Juan Pablo
HU-12	Asignar historias a un sprint Como equipo queremos asignar historias al sprint para comprometer el trabajo de la iteración. – CA-12.1 sólo historias estimadas, con CA y no Hechas – CA-12.2 no se asignan historias a un sprint cerrado; una historia está en un solo sprint abierto – CA-12.3 advertencia (no bloqueo) si los SP comprometidos superan la velocidad del equipo (HU-31)	5	A	1	Juan Pablo
HU-13	Registrar historias completadas Como Product Builder quiero marcar una historia del sprint como completada para registrar el avance. – CA-13.1 sólo historias del sprint activo – CA-13.2 se guarda la fecha de completado – CA-13.3 exige los CA cumplidos (regla de HU-08)	2	A	1	Juan Pablo
HU-14	Cerrar sprint Como Agile Enabler quiero cerrar el sprint para consolidar sus resultados. – CA-14.1 sólo se cierra el sprint activo – CA-14.2 las historias no completadas vuelven al backlog como Pendientes – CA-14.3 se guardan los SP planificados y completados (foto del cierre) – CA-14.4 un sprint cerrado no se reabre ni se modifica	5	A	2	Juan Pablo
HU-15	Consultar sprints anteriores Como integrante quiero consultar los sprints anteriores para analizar cómo trabajamos. – CA-15.1 lista ordenada por número con goal, fechas, SP planificados/completados y % de cumplimiento – CA-15.2 acceso al detalle de cada sprint	2	M	2	Juan Pablo

E4 Estimación

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-16	Estimar historia con story points Como equipo queremos asignar story points a una historia para dimensionar su esfuerzo relativo. – CA-16.1 sólo valores de la escala acordada (Fibonacci: 0, 1, 2, 3, 5, 8, 13, 21) [A CONFIRMAR] – CA-16.2 un valor fuera de la escala devuelve error – CA-16.3 re-estimar conserva el valor anterior en el historial	2	A	1	Mariano

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-17	Abrir sesión de Planning Poker Como Agile Enabler quiero abrir una sesión de Planning Poker para una historia para que el equipo la estime en conjunto. - CA-17.1 sólo para historias no Hechas - CA-17.2 una única sesión abierta por historia - CA-17.3 participan los integrantes activos del proyecto	3	A	2	Mariano
HU-18	Votar de forma oculta Como integrante quiero emitir mi estimación sin que los demás la vean para evitar el efecto ancla. - CA-18.1 un voto por integrante por ronda; se puede cambiar antes del cierre - CA-18.2 los demás sólo ven quién votó, nunca el valor - CA-18.3 la API tampoco expone los votos antes de revelar - CA-18.4 voto fuera de escala → error	5	A	2	Mariano
HU-19	Revelar estimaciones Como equipo queremos ver todas las estimaciones al finalizar la votación para discutir las. - CA-19.1 se revela cuando todos votaron o cuando el Agile Enabler cierra la ronda - CA-19.2 se muestra cada voto con su autor - CA-19.3 una ronda sin votos no se puede revelar	3	A	2	Mariano
HU-20	Detectar diferencias entre estimaciones Como equipo queremos que el sistema marque las diferencias entre estimaciones para enfocar la discusión. - CA-20.1 hay discrepancia si los votos extremos están separados por más de una posición en la escala [A CONFIRMAR] - CA-20.2 se destacan el voto más alto y el más bajo - CA-20.3 si todos coinciden, se sugiere ese valor como consenso	3	A	3	Mariano
HU-21	Realizar nuevas rondas Como Agile Enabler quiero iniciar una nueva ronda de votación para buscar consenso tras la discusión. - CA-21.1 sólo después de revelar - CA-21.2 los votos de rondas anteriores quedan en el historial - CA-21.3 la nueva ronda empieza sin votos y con número correlativo	3	M	3	Mariano
HU-22	Registrar la estimación acordada Como Agile Enabler quiero registrar el valor acordado para que quede como estimación de la historia. - CA-22.1 el valor pertenece a la escala - CA-22.2 se actualizan los SP de la historia - CA-22.3 la sesión queda cerrada y consultable	2	A	3	Mariano

E5 Esfuerzo

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-23	Registrar horas estimadas Como Product Builder quiero registrar las horas estimadas de una historia para compararlas luego con el esfuerzo real. - CA-23.1 horas mayores a 0, admite decimales (1,5) - CA-23.2 si la historia tiene tareas, las horas estimadas son la suma de sus tareas	2	A	1	Carolina
HU-24	Registrar esfuerzo realizado Como integrante quiero registrar fecha, actividad y horas trabajadas en una historia o tarea para medir el esfuerzo real. - CA-24.1 horas mayores a 0 y hasta 24 por registro; el total diario de un integrante no supera 24 h - CA-24.2 la fecha no es futura y está dentro del proyecto - CA-24.3 el integrante pertenece al proyecto	3	A	1	Carolina

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-25	Descomponer historia en tareas Como Product Builder quiero dividir una historia en tareas con responsable y horas estimadas para organizar el trabajo. - CA-25.1 cada tarea tiene título, responsable y horas estimadas - CA-25.2 una tarea pertenece a una sola historia - CA-25.3 no se modifica la estimación de una tarea completada	3	M	2	Carolina
HU-26	Consultar y corregir registros de esfuerzo Como integrante quiero ver y corregir mis registros de horas para mantener datos confiables. - CA-26.1 lista filtrable por integrante, historia y sprint - CA-26.2 cada integrante edita o elimina sólo sus registros (el Agile Enabler, todos) - CA-26.3 muestra horas estimadas vs reales por historia	3	M	2	Carolina

E6 Defectos

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-27	Registrar defecto Como integrante quiero registrar un defecto con descripción, severidad, historia relacionada y sprint de detección para darle seguimiento. - CA-27.1 descripción obligatoria; severidad Crítica, Alta, Media o Baja - CA-27.2 estado inicial Abierto - CA-27.3 la historia y el sprint deben pertenecer al proyecto	3	A	2	Carolina
HU-28	Gestionar el estado de un defecto Como integrante quiero actualizar el estado de un defecto para registrar su resolución. - CA-28.1 transiciones: Abierto → En progreso → Resuelto → Cerrado, y Resuelto → Reabierto - CA-28.2 al resolver se exige sprint de resolución, igual o posterior al de detección - CA-28.3 reabrir limpia el sprint de resolución	3	A	2	Carolina
HU-29	Consultar defectos Como integrante quiero consultar los defectos para priorizar su corrección. - CA-29.1 filtros por severidad, estado y sprint - CA-29.2 conteo por severidad	2	M	2	Carolina

E7 Métricas

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-30	Story points planificados y completados Como Agile Enabler quiero ver los SP planificados y completados por sprint y del proyecto para medir el cumplimiento. - CA-30.1 planificados = suma de SP asignados al sprint; completados = suma de SP de historias Hechas - CA-30.2 historias sin estimar suman 0 y se muestra una advertencia - CA-30.3 totales del proyecto = suma de los sprints	3	A	1	Carolina
HU-31	Velocidad del equipo Como Agile Enabler quiero conocer la velocidad del equipo para decidir cuántos SP comprometer en el próximo sprint. CA-31.1 ver sección 5 (ejemplo completo de trazabilidad)	3	A	1	Carolina
HU-32	Porcentaje de historias completadas Como Agile Enabler quiero ver el porcentaje de historias completadas para evaluar el avance. - CA-32.1 completadas / planificadas × 100, con un decimal - CA-32.2 sprint sin historias → 0 % sin error (división por cero controlada) - CA-32.3 se calcula por sprint y para el proyecto	2	A	1	Carolina

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-33	Horas estimadas, reales y desviación Como Agile Enabler quiero comparar horas estimadas y reales para mejorar nuestras estimaciones. - CA-33.1 desviación = reales – estimadas; desviación % = (reales – estimadas) / estimadas × 100 - CA-33.2 con estimadas = 0 la desviación % se informa como "no aplicable" - CA-33.3 se calcula por historia, sprint y proyecto	3	A	3	Carolina
HU-34	Defectos detectados y resueltos Como Agile Enabler quiero conocer los defectos detectados y resueltos para evaluar la calidad del producto. - CA-34.1 detectados por sprint de detección y resueltos por sprint de resolución - CA-34.2 totales del proyecto y filtro por severidad	2	A	3	Carolina

E8 Dashboard

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-35	Panel de indicadores del proyecto Como Agile Enabler quiero un dashboard con el estado del proyecto para tomar decisiones rápidamente. - CA-35.1 tarjetas con estado, sprint activo, velocidad, % completado, desviación de horas y defectos abiertos - CA-35.2 un proyecto sin datos muestra un estado vacío, sin errores	5	A	3	Carolina
HU-36	Gráficos de métricas Como equipo queremos ver gráficos de las métricas para detectar tendencias. - CA-36.1 velocidad por sprint, SP planificados vs completados, horas estimadas vs reales, defectos por severidad y burndown del sprint activo - CA-36.2 los datos salen de un endpoint JSON con tests	5	A	3	Carolina

E9 Reportes

ID	Historia y criterios de aceptación (borrador)	SP	Pri	Spr	Resp.
HU-37	Reporte de sprint Como Agile Enabler quiero un reporte de un sprint para presentarlo en la Sprint Review. - CA-37.1 incluye historias planificadas y completadas, estimaciones, esfuerzo, métricas y defectos del sprint - CA-37.2 reutiliza los cálculos de <code>internal/metrics</code> - CA-37.3 sprint inexistente → error 404	5	A	3	Juan Pablo
HU-38	Reporte de proyecto Como Agile Enabler quiero un reporte consolidado del proyecto para rendir cuentas al cliente. - CA-38.1 consolida todos los sprints con el mismo contenido que el reporte de sprint - CA-38.2 incluye la evolución de la velocidad	3	A	3	Juan Pablo
HU-39	Exportar reporte a PDF Como Agile Enabler quiero exportar el reporte a PDF para compartirlo fuera del sistema. - CA-39.1 descarga un .pdf llamado proyecto-sprint-fecha.pdf - CA-39.2 el contenido coincide con el reporte en pantalla - CA-39.3 se genera en menos de 10 segundos (límite de la función serverless)	5	A	4	Juan Pablo

6.1 Carga por integrante y por sprint (story points)

Integrante	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Total SP
Juan Pablo	21	12	8	5	46
Mariano	15	21	11	0	47
Carolina	16	17	15	0	48
Equipo	52	50	34	5	141

El Sprint 4 tiene poca carga de desarrollo a propósito: está reservado para pulido, corrección de defectos, documentación, informe final y la presentación (sección 7). La carga real se ajusta en cada Sprint Planning según la velocidad medida.

7. Plan por sprint y por integrante

Sprint 0 — La Preparación (28/09 – 04/10)

Objetivo: formar el equipo, asignar roles, configurar el entorno y construir el Product Backlog inicial junto al cliente.

Integrante	Tareas del Sprint 0
Juan Pablo	- Configurar GitHub Projects: vistas Tablero (Backlog · Sprint Backlog · En progreso · En revisión · Hecho), Tabla de backlog y Roadmap; campos Épica, Story Points, Prioridad, Iteración (Sprint 1–4) y Responsable. - Labels (epica:*, tipo:historia tecnica defecto, prioridad:*), plantillas de issue (HU, defecto) y de PR con checklist de trazabilidad. - Proteger main (PR obligatorio, 1 aprobación, CI verde) y deshabilitar squash merge. - Cargar como issues las historias de E1, E3 y E9. Redactar DoR / DoD en docs/scrum/. - Organizar la reunión inicial con el profesor y la primera Planning.
Mariano	- Crear los proyectos Supabase (dev y prod) y compartir el acceso al equipo. - Proponer el modelo de dominio / diagrama entidad-relación (lo cierran los tres). - Escribir la plantilla SDD (specs/_plantilla.md) con las 8 secciones obligatorias. - Cargar como issues las historias de E2 y E4. Investigar Supabase Auth + validación de JWT en Go (spike).
Carolina	- Crear el proyecto en Vercel vinculado al repo y probar un hello world en Go (spike de TEC-04). - Escribir la plantilla BDD (features/_plantilla.feature) y un ejemplo de godog funcionando. - Redactar docs/metricas.md con la definición y fórmula de cada métrica (glosario compartido). - Cargar como issues las historias de E5, E6, E7 y E8.
Los tres	- Instalar Go, Git y VS Code (extensión Go). Configurar Git con la cuenta institucional (sección 9). - Participar en la reunión con el cliente y validar las historias. Estimar el backlog con Planning Poker (con cartas o una app) — sirve de práctica para la HU-18. - Acordar horarios de Daily y canal de comunicación.

Sprint 1 — El MVP (05/10 – 11/10)

Objetivo: Entregar una versión funcional básica, desplegada, con dominio, persistencia y vistas mínimas. Se realizan todas las ceremonias: Planning, Daily, Review y Retrospective.

Demo esperada en la Review: Crear un proyecto con integrantes, cargar y estimar historias, crear un sprint, asignar historias, marcarlas completadas, registrar horas y ver SP planificados/completados, velocidad y % completado.

Integrante	Ítems del backlog	Además
Juan Pablo	TEC-01, TEC-02, HU-01, HU-03, HU-11, HU-12, HU-13	Facilitar Planning, Dailies, Review y Retro; acta de retro del Sprint 1.
Mariano	TEC-03, HU-05, HU-06, HU-07, HU-08, HU-16	Migración inicial y repositorios de proyecto, historia y sprint (usados por los tres).
Carolina	TEC-04, HU-23, HU-24, HU-30, HU-31, HU-32	Funciones de internal/metrics con TDD usando structs, sin esperar a la BD.

Sprint 2 — La Interfaz (12/10 – 18/10)

Objetivo: Dotar al software de una interfaz usable: login, navegación completa, cierre de sprint, Planning Poker con votos ocultos, tareas, esfuerzo y defectos.

Demo esperada en la Review: Un equipo inicia sesión, estima una historia con Planning Poker (votar y revelar), cierra un sprint y registra un defecto.

Integrante	Ítems del backlog	Además
Juan Pablo	HU-02, HU-04, HU-14, HU-15	Vistas de proyecto y sprints sobre el layout de Carolina; acta de retro.
Mariano	TEC-05, HU-09, HU-10, HU-17, HU-18, HU-19	Vistas de backlog y Planning Poker con polling de HTMX.
Carolina	TEC-07, HU-25, HU-26, HU-27, HU-28, HU-29	Layout base primero (desbloquea las vistas de los demás).

Sprint 3 — Funcionalidad y Calidad (19/10 – 25/10)

Objetivo: Completar funcionalidades clave, implementar la visualización (dashboard y gráficos) y robustecer el sistema (cobertura, integración, manejo de errores).

Demo esperada en la Review: Dashboard con gráficos reales, Planning Poker completo con rondas y diferencias, reportes en pantalla, cobertura $\geq 80\%$ en dominio.

Integrante	Ítems del backlog	Además
Juan Pablo	HU-37, HU-38	Reportes reutilizando internal/metrics; acta de retro; empezar la presentación.
Mariano	TEC-06, HU-20, HU-21, HU-22	Tests de integración, umbral de cobertura en CI y revisión de manejo de errores.
Carolina	HU-33, HU-34, HU-35, HU-36	Endpoint JSON del dashboard con tests; gráficos con Chart.js.

Sprint 4 — El Cierre y la Entrega Final (26/10 – 01/11)

Objetivo: pulir el producto, documentar y preparar la entrega. El sistema permite **exportar un informe del proyecto a PDF**. Cierra con la **Sprint Review Final**: presentación formal del producto al cliente.

Integrante	Tareas del Sprint 4
Juan Pablo	- HU-39 Exportar reporte a PDF. - Compilar Product Backlog, Sprint Backlogs y actas (entregables 2, 3 y 11). - Armar la presentación final y el guion de la demo de trazabilidad (HU-31). - Facilitar la última Review y Retro.
Mariano	Documentación técnica: arquitectura, modelo de datos, API, cómo correr tests y cómo desplegar. - Corrección de defectos abiertos de E2/E4 y endurecimiento de validaciones. - Revisar que toda spec tenga su feature y sus tests (auditoría de trazabilidad).
Carolina	Manual breve de usuario con capturas. Informe de métricas y cobertura (cobertura por paquete, métricas del propio proyecto, evolución de la velocidad). - Corrección de defectos abiertos de E5–E8.
Los tres	- Ensayar la demo; cada uno presenta sus épicas y justifica sus decisiones. - Congelar el código (tag v1.0.0) y verificar el deploy de producción.

Ceremonias (facilita el Agile Enabler)

Ceremonia	Cuándo	Resultado registrado
Sprint Planning	Inicio de cada sprint	Sprint Goal + historias en la iteración del tablero + docs/scrum/sprint-N/planning.md
Daily	3 veces por semana (15 min, puede ser por mensaje)	Qué hice / qué haré / impedimentos → docs/scrum/sprint-N/dailies.md
Sprint Review	Fin de sprint, con el profesor	Demo en producción + feedback → review.md; nuevas historias al backlog
Retrospective	Después de la Review	Acta (Qué salió bien / Qué mejorar / Acciones con responsable) → retro.md

8. Entregables y responsables

#	Entregable	Dónde / cómo	Responsable
1	Repositorio Git con historial de contribuciones	GitHub; cada uno commitea con su cuenta	Los tres
2	Tablero Scrum	GitHub Projects	Juan Pablo
3	Product Backlog y Sprint Backlogs	Vistas por iteración del tablero + export en docs/scrum/	Juan Pablo
4	Especificaciones SDD	/specs	Cada dueño de historia
5	Escenarios BDD	/features (godog)	Cada dueño de historia
6	Pruebas automatizadas	*_test.go + CI	Cada dueño; CI: Juan Pablo / Mariano
7	Código fuente en Go	/internal, /api, /cmd	Los tres
8	Evidencias de TDD	Commits RED / GREEN / REFACTOR (sin squash)	Los tres
9	Software funcional	Producción en Vercel	Carolina (deploy) + los tres
10	Informe de métricas y cobertura	docs/informe-metricas-cobertura.md + PDF	Carolina
11	Actas de retrospectivas	docs/scrum/sprint-N/retro.md	Juan Pablo
12	Documentación técnica y manual de usuario	docs/arquitectura.md, docs/manual-usuario.md	Mariano / Carolina
13	Presentación y demostración final	Diapositivas + demo de HU-31 de punta a punta	Juan Pablo coordina; los tres presentan

8.1 Ideas que suman (valor agregado)

- **Usar nuestra propia app para gestionar el TPI** desde el Sprint 2 (dogfooding): en la presentación final mostramos nuestras métricas reales (velocidad, desviación de horas, defectos) calculadas por el sistema.
- **Métricas extra de bajo costo:** burndown del sprint, precisión de estimación (SP completados / planificados) y densidad de defectos por historia.
- **Registro de uso de IA** (docs/ia/registro.md): qué se pidió, qué se aceptó, qué se corrigió y cómo se validó. Demuestra lo que exige la consigna sobre comprensión y validación.
- **Badge de cobertura y de CI en el README:** evidencia visible de calidad para los profesores.
- **Datos de ejemplo** (make seed) para que la demo y el manual de usuario tengan un proyecto cargado.

9. Configuración de Git con la cuenta institucional

Para que el historial de contribuciones cuente a nombre de cada uno, **cada integrante** configura en su clon del repo su nombre y el email de su cuenta institucional (sólo para este repositorio, sin tocar la configuración global):

```
git clone https://github.com/<cuenta-institucional>/<repo>.git
cd <repo>
git config user.name "Apellido, Nombre"
git config user.email "<email-de-la-cuenta-institucional>"
```

Si en la misma PC ya hay otra cuenta de GitHub iniciada (caso de Juan Pablo), la CLI de GitHub permite tener las dos y cambiar entre ellas: `gh auth login` (agrega la institucional), `gh auth switch` (cambia de cuenta) y `gh auth setup-git` (usa esa sesión para `git push`).

10. Preguntas abiertas [A CONFIRMAR]

Las respuestas pueden cambiar la asignación, el alcance o la tecnología. Conviene cerrarlas antes de la primera Planning.

#	Tema	Pregunta	Propuesta por defecto	Quién responde
1	Roles	¿Quién es el Agile Enabler?	Resuelto: Juan Pablo	—
2	Roles	¿Les parece bien la división por épicas?	La de la sección 3	Equipo
3	Cronograma	¿Fecha exacta de la presentación final? ¿Las Reviews son en clase?	Sprints de 1 semana desde el lun 28/09; cierre dom 01/11	Profesores
4	Visión	¿El profesor ya presentó la visión del producto? ¿Agregó algún requisito?	Sólo lo del PDF	Profesores
5	Alcance	¿Multiusuario con login o un único usuario que carga todo?	Resuelto: multiusuario con login	—
6	Alcance	¿Integrantes del proyecto = usuarios del sistema?	Sí, 1 usuario ↔ 1 integrante	Equipo
7	Estimación	¿Qué escala de story points usamos?	Fibonacci 0–21 (+ "?")	Profesores
8	Estimación	¿Qué se considera "diferencia" en Planning Poker?	Votos extremos a más de 1 posición de distancia	Profesores
9	Esfuerzo	¿Horas estimadas por historia, por tarea o ambas?	Ambas (historia = suma de tareas)	Profesores
10	Métricas	¿Velocidad sobre cuántos sprints?	Últimos 3 sprints cerrados	Profesores
11	Interfaz	¿Go + HTMX o un frontend separado (React/Next.js)?	Resuelto: Go + HTMX + Bootstrap	—
12	Deploy	¿Vercel es obligatorio o puede ser otro hosting para el backend Go?	Vercel; plan B Render/Fly.io	Equipo
13	Calidad	¿Hay un mínimo de cobertura exigido?	80 % en dominio y métricas	Profesores
14	Entrega	¿Formato de la documentación (Markdown en el repo, PDF, Word)?	Markdown + PDF exportado	Profesores
15	Git	¿Mariano, Carolina y los profesores tienen acceso al repo?	Resuelto: sí	—